Chapter 6

Semantic Analysis

6.1 Attributes and Attribute Grammars
6.2 Algorithms for Attribute Computation
6.3 The Symbol Table
6.4 Data Types and Type Checking
6.5 A Semantic Analyzer for the TINY Language

In this chapter we investigate the phase of the compiler that computes additional information needed for compilation once the syntactic structure of a program is known. This phase is referred to as semantic analysis because it involves computing information that is beyond the capabilities of context-free grammars and standard parsing algorithms and, therefore, is not regarded as syntax.[1] The information computed is also closely related to the eventual meaning, or semantics, of the program being translated. Since the analysis performed by a compiler is by definition static (it takes place prior to execution), such semantic analysis is also called **static semantic analysis**. In a typical statically typed language such as C, semantic analysis involves building a symbol table to keep track of the meanings of names established in declarations and performing type inference and type checking on expressions and statements to determine their correctness within the type rules of the language.

Semantic analysis can be divided into two categories. The first is the analysis of a program required by the rules of the programming language to establish its correctness and to guarantee proper execution. The amount of such analysis required by a language definition varies tremendously from language to language. In dynamically oriented languages such as LISP and Smalltalk, there may be no static semantic analysis at all, while in a language such as Ada there are strong requirements that a program must meet to be executable. Other languages fall in between these extremes (Pascal, for instance, being not quite as strict in its static requirements as Ada and C being not quite as permissive as LISP).

1. This point was discussed in some detail in Section 3.6.3 of Chapter 3.

The second category of semantic analysis is the analysis performed by a compiler to enhance the efficiency of execution of the translated program. This kind of analysis is usually included in discussions of "optimization," or code improving techniques. We will survey some of these methods in the chapter on code generation, while in this chapter we focus on the common analyses that are required by a language definition for correctness. It should be noted that the techniques studied here apply to both situations. Also, the two categories are not mutually exclusive, since correctness requirements, such as static type checking, also allow a compiler to generate more efficient code than for a language without these requirements. In addition, it is worth noting that the correctness requirements discussed here can never establish the complete correctness of a program, but only a kind of partial correctness. Such requirements are still useful in that they provide the programmer with information to improve the security and robustness of the program.

Static semantic analysis involves both the **description** of the analyses to perform and the **implementation** of the analyses using appropriate algorithms. In this, it is similar to lexical and syntactic analysis. In syntactic analysis, for example, we used context-free grammars in Backus-Naus Form (BNF) to describe the syntax and various top-down and bottom-up parsing algorithms to implement the syntax. In semantic analysis, the situation is not as clear, partially because there is no standard method (like BNF) used to specify the static semantics of a language and partially because the amount and kind of static semantic analysis varies so widely from language to language. One method of describing semantic analysis that is frequently used to good effect by compiler writers is to identify **attributes**, or properties, of language entities that must be computed and to write **attribute equations**, or **semantic rules**, that express how the computation of such attributes is related to the grammar rules of the language. Such a set of attributes and equations is called an **attribute grammar**. Attribute grammars are most useful for languages that obey the principle of **syntax-directed semantics**, which asserts that the semantic content of a program is closely related to its syntax. All modern languages have this property. Unfortunately, the compiler writer must usually construct an attribute grammar by hand from the language manual, since it is rarely given by the language designer. Worse yet, the construction of an attribute grammar may be made unnecessarily complicated by its very adherence to the explicit syntactic structure of the language. A much better basis for the expression of semantic computations is the abstract syntax, as represented by an abstract syntax tree. Yet the specification of the abstract syntax is also usually left to the compiler writer by the language designer.

Algorithms for the implementation of semantic analysis are also not as clearly expressible as parsing algorithms. Again, this is partially due to the same problems that were just mentioned regarding the specification of semantic analysis. There is, however, an additional problem caused by the timing of the analysis during the compilation process. If semantic analysis can be suspended until all the syntactic analysis (and the construction of an abstract syntax tree) is complete, then the task of implementing semantic analysis is made considerably easier and consists essentially of specifying an order for a traversal of the syntax tree, together with the computations to be performed each time a node is encountered in the traversal. This, how-

ever, implies that the compiler must be multipass. If, on the other hand, it is necessary for the compiler to perform all its operations in a single pass (including code generation), then the implementation of semantic analysis becomes much more an ad hoc process of finding a correct order and method for computing semantic information (assuming such a correct order actually exists). Fortunately, modern practice increasingly permits the compiler writer to use multiple passes to simplify the processes of semantic analysis and code generation.

Despite this somewhat disorderly state of semantic analysis, it is extremely useful to study attribute grammars and specification issues, since this will pay off in an ability to write clearer, more concise, less error-prone code for semantic analysis, as well as permit an easier understanding of that code.

The chapter begins, therefore, with a study of attributes and attribute grammars. It continues with techniques for implementing the computations specified by an attribute grammar, including the inferring of an order for the computations and accompanying tree traversals. Two subsequent sections concentrate on the major areas of semantic analysis: symbol tables and type checking. The last section describes a semantic analyzer for the TINY programming language introduced in previous chapters.

Unlike previous chapters, this chapter contains no description of a **semantic analyzer generator** or general tool for constructing semantic analyzers. Though a number of such tools have been constructed, none has achieved the wide use and availability of Lex or Yacc. In the Notes and References section at the end of the chapter, we mention a few of these tools and provide references to the literature for the interested reader.

6.1 ATTRIBUTES AND ATTRIBUTE GRAMMARS

An **attribute** is any property of a programming language construct. Attributes can vary widely in the information they contain, their complexity, and particularly the time during the translation/execution process when they can be determined. Typical examples of attributes are

- The data type of a variable
- The value of an expression
- The location of a variable in memory
- The object code of a procedure
- The number of significant digits in a number

Attributes may be fixed prior to the compilation process (or even the construction of a compiler). For instance, the number of significant digits in a number may be fixed (or at least given a minimum value) by the definition of a language. Also, attributes may be only determinable during program execution, such as the value of a (nonconstant) expression, or the location of a dynamically allocated data structure. The process of computing an attribute and associating its computed value with the language construct in question is referred to as the **binding** of the attribute. The time during the compila-

tion/execution process when the binding of an attribute occurs is called its **binding time**. Binding times of different attributes vary, and even the same attribute can have quite different binding times from language to language. Attributes that can be bound prior to execution are called **static**, while attributes that can only be bound during execution are **dynamic**. A compiler writer is, of course, interested in those static attributes that are bound at translation time.

Consider the previously given sample list of attributes. We discuss the binding time and significance during compilation of each of the attributes in the list.

- In a statically typed language like C or Pascal, the data type of a variable or expression is an important compile-time attribute. A **type checker** is a semantic analyzer that computes the data type attribute of all language entities for which data types are defined and verifies that these types conform to the type rules of the language. In a language like C or Pascal, a type checker is an important part of semantic analysis. In a language like LISP, however, data types are dynamic, and a LISP compiler must generate code to compute types and perform type checking during program execution.
- The values of expressions are usually dynamic, and a compiler will generate code to compute their values during execution. However, some expressions may, in fact, be constant (3 + 4 * 5, for example), and a semantic analyzer may choose to evaluate them during compilation (this process is called **constant folding**).
- The allocation of a variable may be either static or dynamic, depending on the language and the properties of the variable itself. For example, in FORTRAN77 all variables are allocated statically, while in LISP all variables are allocated dynamically. C and Pascal have a mixture of static and dynamic variable allocation. A compiler will usually put off computations associated with allocation of variables until code generation, since such computations depend on the runtime environment and, occasionally, on the details of the target machine. (The next chapter treats this in more detail.)
- The object code of a procedure is clearly a static attribute. The code generator of a compiler is wholly concerned with the computation of this attribute.
- The number of significant digits in a number is an attribute that is often not explicitly treated during compilation. It is implicit in the way the compiler writer implements representations for values, and this is usually considered as part of the runtime environment, discussed in the next chapter. However, even the scanner may need to know the number of allowable significant digits if it is to translate constants correctly.

As we see from these examples, attribute computations are extremely varied. When they appear explicitly in a compiler, they may occur at any time during compilation: even though we associate attribute computation most strongly with the semantic analysis phase, both the scanner and the parser may need to have attribute information available to them, and some semantic analysis may need to be performed at the same time as parsing. In this chapter, we focus on typical computations that occur prior to code generation and after parsing (but see Section 6.2.5 for information on semantic analy-

sis during parsing). Attribute analysis that directly applies to code generation will be discussed in Chapter 8.

6.1.1 Attribute Grammars

In **syntax-directed semantics**, attributes are associated directly with the grammar symbols of the language (the terminals and nonterminals).[2] If X is a grammar symbol, and a is an attribute associated to X, then we write $X.a$ for the value of a associated to X. This notation is reminiscent of a record field designator of Pascal or (equivalently) a structure member operation of C. Indeed, a typical way of implementing attribute calculations is to put attribute values into the nodes of a syntax tree using record fields (or structure members). We will see this in more detail in the next section.

Given a collection of attributes $a_1, \ldots, a_k$, the principle of syntax-directed semantics implies that for each grammar rule $X_0 \to X_1\ X_2 \ldots X_n$ (where X_0 is a nonterminal and the other X_i are arbitrary symbols), the values of the attributes $X_i.a_j$ of each grammar symbol X_i are related to the values of the attributes of the other symbols in the rule. Should the same symbol X_i appear more than once in the grammar rule, then each occurrence must be distinguished from the other occurrences by suitable subscripting, so that the attribute values of different occurrences may be distinguished. Each relationship is specified by an **attribute equation** or **semantic rule**[3] of the form

$$X_i.a_j = f_{ij}(X_0.a_1, \ldots, X_0.a_k, X_1.a_1, \ldots, X_1.a_k, \ldots, X_n.a_1, \ldots, X_n.a_k)$$

where f_{ij} is a mathematical function of its arguments. An **attribute grammar** for the attributes $a_1, \ldots, a_k$ is the collection of all such equations, for all the grammar rules of the language.

In this generality, attribute grammars may appear to be extremely complex. In practice the functions f_{ij} are usually quite simple. Also, it is rare for attributes to depend on large numbers of other attributes, and so attributes can be separated into small independent sets of interdependent attributes, and attribute grammars can be written separately for each set.

Typically, attribute grammars are written in tabular form, with each grammar rule listed with the set of attribute equations, or semantic rules associated to that rule, as follows:[4]

---

2. Syntax-directed semantics could just as easily be called **semantics-directed syntax**, since in most languages the syntax is designed with the (eventual) semantics of the construct already in mind.

3. In later work we will view semantic rules as being somewhat more general than attribute equations. For now, the reader can consider them to be identical.

4. We will always use the heading "semantic rules" in these tables instead of "attribute equations" to allow for a more general interpretation of semantic rules later on.

Grammar Rule	Semantic Rules
Rule 1	Associated attribute equations
.	.
.	.
.	.
Rule *n*	Associated attribute equations

We proceed immediately to several examples.

Example 6.1 Consider the following simple grammar for unsigned numbers:

$$number \rightarrow number\ digit \mid digit$$
$$digit \rightarrow \mathbf{0} \mid \mathbf{1} \mid \mathbf{2} \mid \mathbf{3} \mid \mathbf{4} \mid \mathbf{5} \mid \mathbf{6} \mid \mathbf{7} \mid \mathbf{8} \mid \mathbf{9}$$

The most significant attribute of a number is its value, to which we give the name *val*. Each digit has a value which is directly computable from the actual digit it represents. Thus, for example, the grammar rule $digit \rightarrow \mathbf{0}$ implies that *digit* has value 0 in this case. This can be expressed by the attribute equation $digit.val = 0$, and we associate this equation with the rule $digit \rightarrow \mathbf{0}$. Furthermore, each number has a value based on the digits it contains. If a number is derived using the rule

$$number \rightarrow digit$$

then the number contains just one digit, and its value is the value of this digit. The attribute equation that expresses this fact is

$$number.val = digit.val$$

If a number contains more than one digit, then it is derived using the grammar rule

$$number \rightarrow number\ digit$$

and we must express the relationship between the value of the symbol on the left-hand side of the grammar rule and the values of the symbols on the right-hand side. Note that the two occurrences of *number* in the grammar rule must be distinguished, since the *number* on the right will have a different value from that of the *number* on the left. We distinguish them by using subscripts, and we write the grammar rule as follows:

$$number_1 \rightarrow number_2\ digit$$

Now consider a number such as **34**. A (leftmost) derivation of this number is as follows: *number* ⇒ *number digit* ⇒ *digit digit* ⇒ **3** *digit* ⇒ **34**. Consider the use of the grammar rule $number_1 \rightarrow number_2\ digit$ in the first step of this derivation. The nonterminal $number_2$ corresponds to the digit **3**, while *digit* corresponds to the digit **4**. The values of each are 3 and 4, respectively. To obtain the value of $number_1$ (that is, 34), we must multiply the value of $number_2$ by 10 and add the value of *digit*: 34 = 3 * 10 + 4. In other words, we shift the 3 one decimal place to the left and add 4. This corresponds to the attribute equation

$$number_1.val = number_2.val * 10 + digit.val$$

A complete attribute grammar for the *val* attribute is given in Table 6.1.

The meaning of the attribute equations for a particular string can be visualized using the parse tree for the string. For example, the parse tree for the number **345** is given in Figure 6.1. In this figure the calculation corresponding to the appropriate attribute equation is shown below each interior node. Viewing the attribute equations as computations on the parse tree is important for algorithms to compute attribute values, as we shall see in the next section.[5]

In both Table 6.1 and Figure 6.1 we have emphasized the difference between the syntactic representation of a digit and the value, or semantic content, of the digit by using different fonts. For instance, in the grammar rule *digit* → **0**, the digit **0** is a token or character, while *digit.val* = 0 means that the digit has the numeric value 0.

Table 6.1
Attribute grammar for Example 6.1

Grammar Rule	Semantic Rules
$number_1 \rightarrow number_2\ digit$	$number_1.val = number_2.val * 10 + digit.val$
number → *digit*	*number.val* = *digit.val*
digit → **0**	*digit.val* = 0
digit → **1**	*digit.val* = 1
digit → **2**	*digit.val* = 2
digit → **3**	*digit.val* = 3
digit → **4**	*digit.val* = 4
digit → **5**	*digit.val* = 5
digit → **6**	*digit.val* = 6
digit → **7**	*digit.val* = 7
digit → **8**	*digit.val* = 8
digit → **9**	*digit.val* = 9

5. In fact, numbers are usually recognized as tokens by the scanner, and their numeric values are easily computed at the same time. In doing so, however, the scanner is likely to use implicitly the attribute equations specified here.

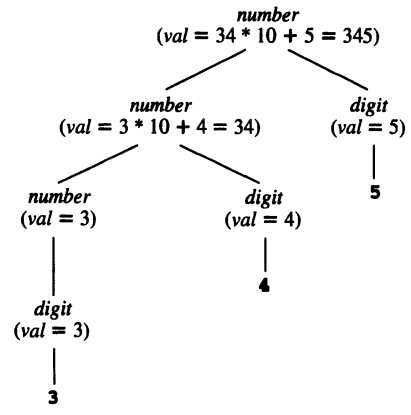


Figure 6.1 Parse tree showing attribute computations for Example 6.1

§

Example 6.2 Consider the following grammar for simple integer arithmetic expressions:

$$exp \rightarrow exp + term \mid exp - term \mid term$$
$$term \rightarrow term * factor \mid factor$$
$$factor \rightarrow (exp) \mid \mathbf{number}$$

This grammar is a slightly modified version of the simple expression grammar studied extensively in previous chapters. The principal attribute of an *exp* (or *term* or *factor*) is its numeric value, which we write as *val*. The attribute equations for the *val* attribute are given in Table 6.2.

Table 6.2 Attribute grammar for Example 6.2

Grammar Rule	Semantic Rules
$exp_1 \rightarrow exp_2 + term$	$exp_1.val = exp_2.val + term.val$
$exp_1 \rightarrow exp_2 - term$	$exp_1.val = exp_2.val - term.val$
$exp \rightarrow term$	$exp.val = term.val$
$term_1 \rightarrow term_2 * factor$	$term_1.val = term_2.val * factor.val$
$term \rightarrow factor$	$term.val = factor.val$
$factor \rightarrow (exp)$	$factor.val = exp.val$
$factor \rightarrow \mathbf{number}$	$factor.val = \mathbf{number}.val$

These equations express the relationship between the syntax of the expressions and the semantics of the arithmetic computations to be performed. Note, for example, the difference between the syntactic symbol + (a token) in the grammar rule

$$exp_1 \rightarrow exp_2 + term$$

and the arithmetic addition operation + to be performed in the equation

$$exp_1.val = exp_2.val + term.val$$

Note also that there is no equation with **number**.*val* on the left-hand side. As we shall see in the next section, this implies that **number**.*val* must be computed prior to any semantic analysis that uses this attribute grammar (e.g., by the scanner). Alternatively, if we want this value to be explicit in the attribute grammar, we must add grammar rules and attribute equations to the attribute grammar (for instance, the equations of Example 6.1).

We can also express the computations implied by this attribute grammar by attaching equations to nodes in a parse tree, as in Example 6.1. For instance, given the expression `(34-3)*42`, we can express the semantics of its value on its parse tree as in Figure 6.2. §

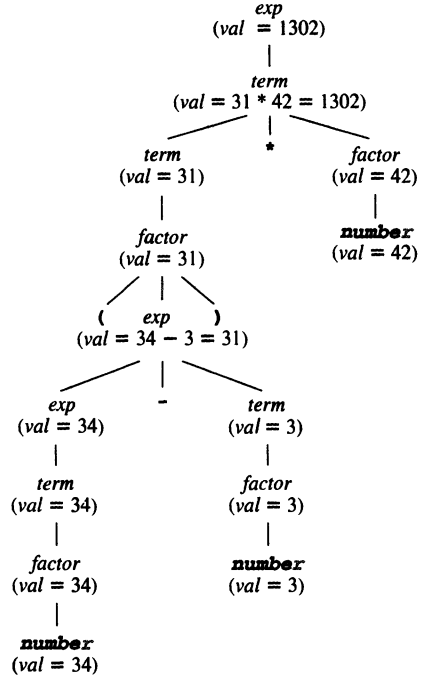


Figure 6.2
Parse tree for `(34-3)*42` showing *val* attribute computations for the attribute grammar of Example 6.2

Example 6.3 Consider the following simple grammar of variable declarations in a C-like syntax:

decl → *type var-list*
type → **int** | **float**
var-list → **id**, *var-list* | **id**

We want to define a data type attribute for the variables given by the identifiers in a declaration and write equations expressing how the data type attribute is related to the type of the declaration. We do this by constructing an attribute grammar for a *dtype* attribute (we use the name *dtype* to distinguish the attribute from the nonterminal *type*). The attribute grammar for *dtype* is given in Table 6.3. We make the following remarks about the attribute equations in that figure.

First, the values of *dtype* are from the set {*integer*, *real*}, corresponding to the tokens **int** and **float**. The nonterminal *type* has a *dtype* given by the token it represents. This *dtype* corresponds to the *dtype* of the entire *var-list*, by the equation associated to the grammar rule for *decl*. Each **id** in the list has this same *dtype*, by the equations associated to *var-list*. Note that there is no equation involving the *dtype* of the nonterminal *decl*. Indeed, a *decl* need not have a *dtype*—it is not necessary for the value of an attribute to be specified for all grammar symbols.

As before, the attribute equations can be displayed on a parse tree. An example is given in Figure 6.3. §

Table 6.3
Attribute grammar for Example 6.3

Grammar Rule	Semantic Rules
decl → *type var-list*	*var-list.dtype* = *type.dtype*
type → **int**	*type.dtype* = *integer*
type → **float**	*type.dtype* = *real*
$var\text{-}list_1$ → **id**, $var\text{-}list_2$	**id**.*dtype* = $var\text{-}list_1$.*dtype* $var\text{-}list_2$.*dtype* = $var\text{-}list_1$.*dtype*
var-list → **id**	**id**.*dtype* = *var-list.dtype*

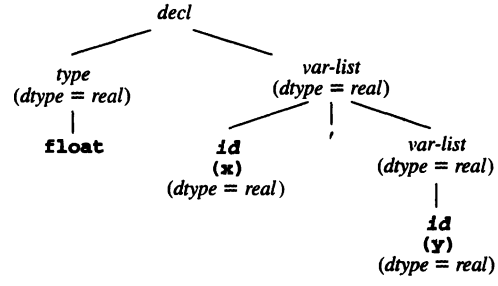


Figure 6.3
Parse tree for the string float x,y showing the dtype attribute as specified by the attribute grammar of Table 6.3.

In the examples so far, there was only one attribute. Attribute grammars may involve several interdependent attributes. The next example is a simple situation where there are interdependent attributes.

Example 6.4

Consider a modification of the number grammar of Example 6.1, where numbers may be octal or decimal. Suppose this is indicated by a one-character suffix **o** (for octal) or **d** (for decimal). Then we have the following grammar:

based-num → *num basechar*
basechar → **o** | **d**
num → *num digit* | *digit*
digit → **0** | **1** | **2** | **3** | **4** | **5** | **6** | **7** | **8** | **9**

In this case *num* and *digit* require a new attribute *base*, which is used to compute the *val* attribute. The attribute grammar for *base* and *val* is given in Table 6.4.

Table 6.4
Attribute grammar for Example 6.4

Grammar Rule	Semantic Rules
based-num → *num basechar*	*based-num.val* = *num.val* *num.base* = *basechar.base*
basechar → **o**	*basechar.base* = 8
basechar → **d**	*basechar.base* = 10
$num_1 \rightarrow num_2\ digit$	$num_1.val$ = **if** *digit.val* = *error* **or** $num_2.val$ = *error* **then** *error* **else** $num_2.val * num_1.base + digit.val$ $num_2.base = num_1.base$ $digit.base = num_1.base$
num → *digit*	*num.val* = *digit.val* *digit.base* = *num.base*
digit → **0**	*digit.val* = 0
digit → **1**	*digit.val* = 1
...	...
digit → **7**	*digit.val* = 7
digit → **8**	*digit.val* = **if** *digit.base* = 8 **then** *error* **else** 8
digit → **9**	*digit.val* = **if** *digit.base* = 8 **then** *error* **else** 9

Two new features should be noted in this attribute grammar. First, the BNF grammar does not itself eliminate the erroneous combination of the (non-octal) digits **8** and **9** with the **o** suffix. For instance, the string **189o** is syntactically correct according to the above BNF, but cannot be assigned any value. Thus, a new *error* value is needed for such cases. Additionally, the attribute grammar must express the fact that the inclusion of **8** or **9** in a number with an **o** suffix results in an *error* value. The easiest way to do this is to use an **if-then-else** expression in the functions of the appropriate attribute equations. For instance, the equation

$num_1.val$ =
if *digit.val* = *error* **or** $num_2.val$ = *error*
then *error*
else $num_2.val * num_1.base + digit.val$

corresponding to the grammar rule $num_1 \rightarrow num_2\ digit$ expresses the fact that if either of $num_2.val$ or *digit.val* are *error* then $num_1.val$ must also be *error*, and only if that is not the case is $num_1.val$ given by the formula $num_2.val * num_1.base + digit.val$.

To conclude this example, we again show the attribute calculations on a parse tree. Figure 6.4 gives a parse tree for the number **345o**, together with the attribute values computed according to the attribute grammar of Table 6.4.

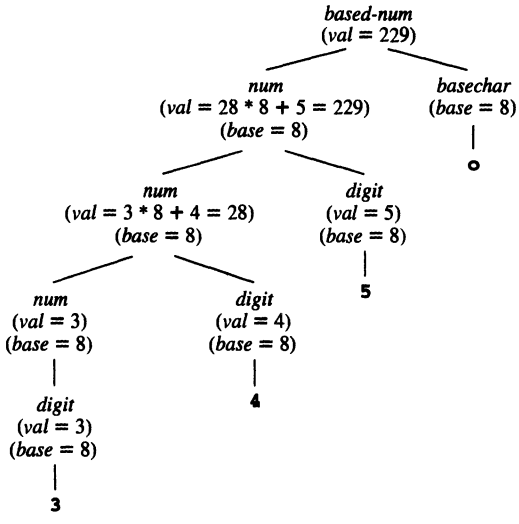


Figure 6.4
Parse tree showing attribute computations for Example 6.4

§

6.1.2 Simplifications and Extensions to Attribute Grammars

The use of an **if-then-else** expression extends the kinds of expressions that can appear in an attribute equation in a useful way. The collection of expressions allowable in an attribute equation is called the **metalanguage** for the attribute grammar. Usually we want a metalanguage whose meaning is clear enough that confusion over its own semantics does not arise. We also want a metalanguage that is close to an actual programming language, since, as we shall see shortly, we want to turn the attribute equations into working code in a semantic analyzer. In this book we use a metalanguage that is limited to arithmetic, logical, and a few other kinds of expressions, together with an **if-then-else** expression, and occassionally a **case** or **switch** expression.

An additional feature useful in specifying attribute equations is to add to the metalanguage the use of functions whose definitions may be given elsewhere. For instance, in the grammars for numbers we have been writing out attribute equations for each of the choices of *digit*. Instead, we could adopt a more concise convention by writing the grammar rule for *digit* as *digit* → **D** (where **D** is understood to be one of the digits) and then writing the corresponding attribute equation as

$$digit.val = numval(\mathbf{D})$$

Here *numval* is a function whose definition must be specified elsewhere as a supplement to the attribute grammar. For instance, we might give the following definition of *numval* as C code:

```
int numval(char D)
{ return (int)D - (int)'0';}
```

A further simplification that can be useful in specifying attribute grammars is to use an ambiguous, but simpler, form of the original grammar. In fact, since the parser is assumed to have already been constructed, all ambiguity will have been dealt with at that stage, and the attribute grammar can be freely based on ambiguous constructs, without implying any ambiguity in the resulting attributes. For instance, the arithmetic expression grammar of Example 6.2 has the following simpler, but ambiguous, form:

$$exp \rightarrow exp + exp \mid exp - exp \mid exp * exp \mid (exp) \mid \texttt{number}$$

Using this grammar, the *val* attribute can be defined by the table in Table 6.5 (compare this with Table 6.2).

Table 6.5
Defining the *val* attribute for an expression using an ambiguous grammar

Grammar Rule	Semantic Rules
$exp_1 \rightarrow exp_2 + exp_3$	$exp_1.val = exp_2.val + exp_3.val$
$exp_1 \rightarrow exp_2 - exp_3$	$exp_1.val = exp_2.val - exp_3.val$
$exp_1 \rightarrow exp_2 * exp_3$	$exp_1.val = exp_2.val * exp_3.val$
$exp_1 \rightarrow (exp_2)$	$exp_1.val = exp_2.val$
$exp \rightarrow$ **number**	$exp.val =$ **number**$.val$

A simplification can also be made in displaying attribute values by using an abstract syntax tree instead of a parse tree. An abstract syntax tree must always have enough structure so that the semantics defined by an attribute grammar can be expressed. For instance, the expression **(34-3)*42**, whose parse tree and *val* attributes were shown in Figure 6.2, can have its semantics completely expressed by the abstract syntax tree of Figure 6.5.

It is not surprising that the syntax tree itself can be specified by an attribute grammar as the next example shows.

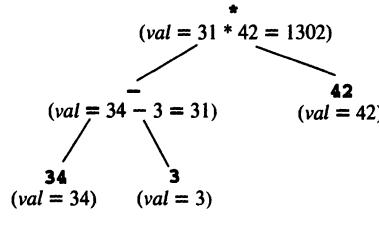


Figure 6.5
Abstract syntax tree for **(34-3)*42** showing *val* attribute computations for the attribute grammar of Table 6.2 or Table 6.5.

Example 6.5

Given the grammar for simple integer arithmetic expressions of Example 6.2 (page 264), we can define an abstract syntax tree for expressions by the attribute grammar given in Table 6.6. In this attribute grammar we have used two auxiliarly functions *mkOpNode* and *mkNumNode*. The *mkOpNode* function takes three parameters (an operator token and two syntax trees) and constructs a new tree node whose operator label is the first parameter, and whose children are the second and third parameters. The *mkNumNode* function takes one parameter (a numeric value) and constructs a leaf node

representing a number with that value. In Table 6.6 we have written the numeric value as **number**.*lexval* to indicate that it is constructed by the scanner. In fact, this could be the actual numeric value of the number, or its string representation, depending on the implementation. (Compare the equations in Table 6.6 with the recursive descent construction of the TINY syntax tree in Appendix B.)

Table 6.6
Attribute grammar for abstract syntax trees of simple integer arithmetic expressions

Grammar Rule	Semantic Rules
$exp_1 \rightarrow exp_2$ + *term*	exp_1.*tree* = *mkOpNode* (+, exp_2.*tree*, *term.tree*)
$exp_1 \rightarrow exp_2$ - *term*	exp_1.*tree* = *mkOpNode*(-, exp_2.*tree*, *term.tree*)
exp → *term*	*exp.tree* = *term.tree*
$term_1 \rightarrow term_2$ * *factor*	$term_1$.*tree* = *mkOpNode*(*, $term_2$.*tree*, *factor.tree*)
term → *factor*	*term.tree* = *factor.tree*
factor → (*exp*)	*factor.tree* = *exp.tree*
factor → **number**	*factor.tree* = *mkNumNode*(**number**.*lexval*)

§

One question that is central to the specification of attributes using attribute grammars is: How can we be sure that a particular attribute grammar is consistent and complete, that is, that it uniquely defines the given attributes? The simple answer is that so far we cannot. This is a question similar to that of determining whether a grammar is ambiguous. In practice, it is the parsing algorithms that we use that determine the adequacy of a grammar, and a similar situation occurs in the case of attribute grammars. Thus, the algorithmic methods for computing attributes that we study in the next section will determine whether an attribute grammar is adequate to define the attribute values.

6.2 ALGORITHMS FOR ATTRIBUTE COMPUTATION

In this section we study the ways an attribute grammar can be used as a basis for a compiler to compute and use the attributes defined by the equations of the attribute grammar. Basically, this amounts to turning the attribute equations into computation rules. Thus, the attribute equation

$$X_i.a_j = f_{ij}(X_0.a_1, \ldots, X_0.a_k, X_1.a_1, \ldots, X_1.a_k, \ldots, X_n.a_1, \ldots, X_n.a_k)$$

is viewed as an assignment of the value of the functional expression on the right-hand side to the attribute $X_i.a_j$. For this to succeed, the values of all the attributes that appear on the right-hand side must already exist. This requirement can be ignored in the specification of attribute grammars, where the equations can be written in arbitrary order without affecting their validity. The problem of implementing an algorithm corre-

sponding to an attribute grammar consists primarily in finding an order for the evaluation and assignment of attributes that ensures that all attribute values used in each computation are available when each computation is performed. The attribute equations themselves indicate the order constraints on the computation of the attributes, and the first task is to make the order constraints explicit by using directed graphs to represent these constraints. These directed graphs are called dependency graphs.

6.2.1 Dependency Graphs and Evaluation Order

Given an attribute grammar, each grammar rule choice has an **associated dependency graph**. This graph has a node labeled by each attribute $X_i.a_j$ of each symbol in the grammar rule, and for each attribute equation

$$X_i.a_j = f_{ij}(\ldots, X_m.a_k, \ldots)$$

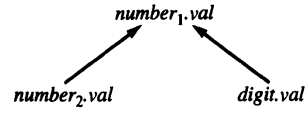
associated to the grammar rule there is an edge from each node $X_m.a_k$ in the right-hand side to the node $X_i.a_j$ (expressing the dependency of $X_i.a_j$ on $X_m.a_k$). Given a legal string in the language generated by the context-free grammar, the **dependency graph** of the string is the union of the dependency graphs of the grammar rule choices representing each (nonleaf) node of the parse tree of the string.

In drawing the dependency graph for each grammar rule or string, the nodes associated to each symbol X are drawn in a group, so that the dependencies can be viewed as structured around a parse tree.

Example 6.6 Consider the grammar of Example 6.1, with the attribute grammar as given in Table 6.1. There is only one attribute, *val*, so for each symbol there is only one node in each dependency graph, corresponding to its *val* attribute. The grammar rule choice $number_1 \rightarrow number_2\ digit$ has the single associated attribute equation

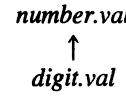
$$number_1.val = number_2.val * 10 + digit.val$$

The dependency graph for this grammar rule choice is



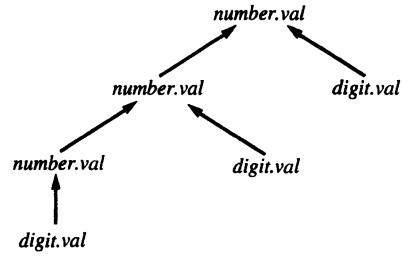
(In future dependency graphs, we will omit the subscripts for repeated symbols, since the graphical representation clearly distinguishes the different occurrences as different nodes.)

Similarly, the dependency graph for the grammar rule $number \rightarrow digit$ with the attribute equation $number.val = digit.val$ is



For the remaining grammar rules of the form *digit* → **D** the dependency graphs are trivial (there are no edges), since *digit.val* is computed directly from the right-hand side of the rule.

Finally, the string **345** has the following dependency graph corresponding to its parse tree (see Figure 6.1):

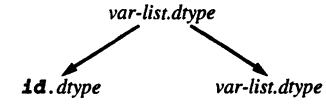


§

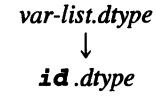
Example 6.7 Consider the grammar of Example 6.3, with the attribute grammar for the attribute *dtype* given in Table 6.3. In this example the grammar rule $var\text{-}list_1 \rightarrow$ **id,** $var\text{-}list_2$ has the two associated attribute equations

$$\textbf{id}.\ dtype = var\text{-}list_1.dtype$$
$$var\text{-}list_2.dtype = var\text{-}list_1.dtype$$

and the dependency graph

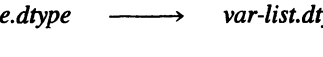


Similarly, the grammar rule *var-list* → **id** has the dependency graph

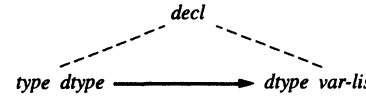


The two rules *type* → **int** and *type* → **float** have trivial dependency graphs.

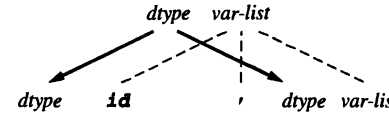
Finally, the rule *decl* → *type var-list* with the associated equation *var-list.dtype* = *type.dtype* has the dependency graph



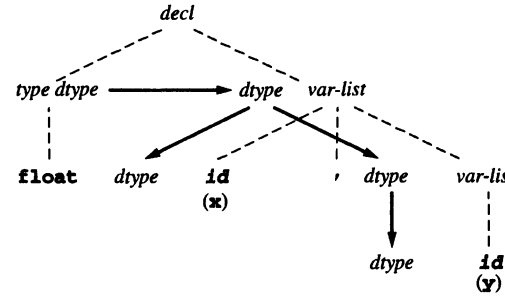
In this case, since *decl* is not directly involved in the dependency graph, it is not completely clear which grammar rule has this graph associated to it. For this reason (and a few other reasons that we discuss later), we often draw the dependency graph superimposed over a parse tree segment corresponding to the grammar rule. Thus, the above dependency graph can be drawn as



and this makes clearer the grammar rule to which the dependency is associated. Note, too, that when we draw the parse tree nodes we suppress the dot notation for the attributes, and represent the attributes of each node by writing them next to their associated node. Thus, the first dependency graph in this example can also be written as



Finally, the dependency graph for the string `float x,y` is



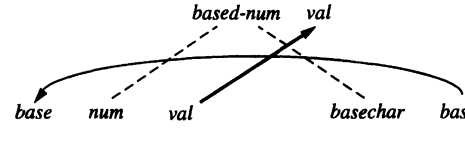
§

Example 6.8 Consider the grammar of based numbers of Example 6.4, with the attribute grammar for the attributes *base* and *val* as given in Table 6.4. We will draw the dependency graphs for the four grammar rules

based-num → *num basechar*
num → *num digit*
num → *digit*
digit → **9**

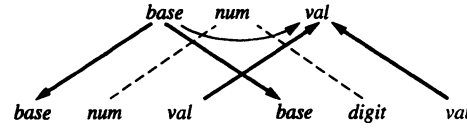
and for the string **345o**, whose parse tree is given in Figure 6.4.

We begin with the dependency graph for the grammar rule *based-num* → *num basechar*:



This graph expresses the dependencies of the two associated equations *based-num.val* = *num.val* and *num.base* = *basechar.base*.

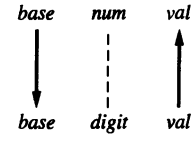
Next we draw the dependency graph corresponding to the grammar rule *num* → *num digit*:



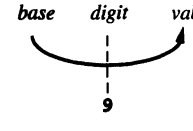
This graph expresses the dependencies of the three attribute equations

$num_1.val =$
  if $digit.val = error$ **or** $num_2.val = error$
  then *error*
  else $num_2.val * num_1.base + digit.val$
$num_2.base = num_1.base$
$digit.base = num_1.base$

The dependency graph for the grammar rule *num* → *digit* is similar:

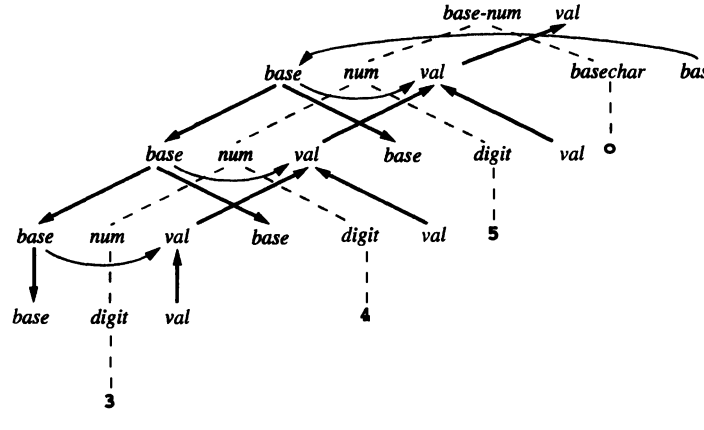


Finally, we draw the dependency graph for the grammar rule *digit* → **9**:



This graph expresses the dependency created by the equation *digit.val* = **if** *digit.base* = 8 **then** *error* **else** 9, namely, that *digit.val* depends on *digit.base* (it is part of the test in the **if**-expression). It remains to draw the dependency graph for the string **345o**. This is done in Figure 6.6.

Figure 6.6
Dependency graph for the string 345o (Example 6.8)



§

Suppose now that we wish to determine an algorithm that computes the attributes of an attribute grammar using the attribute equations as computation rules. Given a particular string of tokens to be translated, the dependency graph of the parse tree of the string gives a set of order constraints under which the algorithm must operate to compute the attributes for that string. Indeed, any algorithm must compute the attribute at each node in the dependency graph *before* it attempts to compute any successor attributes. A traversal order of the dependency graph that obeys this restriction is called a **topological sort**, and a well-known necessary and sufficient condition for a topological sort to exist is that the graph must be **acyclic**. Such graphs are called **directed acyclic graphs**, or **DAGs**.

Example 6.9

The dependency graph of Figure 6.6 is a DAG. In Figure 6.7 we number the nodes of the graph (and erase the underlying parse tree for ease of visualization). One topological sort is given by the node order in which the nodes are numbered. Another topological sort is given by the order

12 6 9 1 2 11 3 8 4 5 7 10 13 14

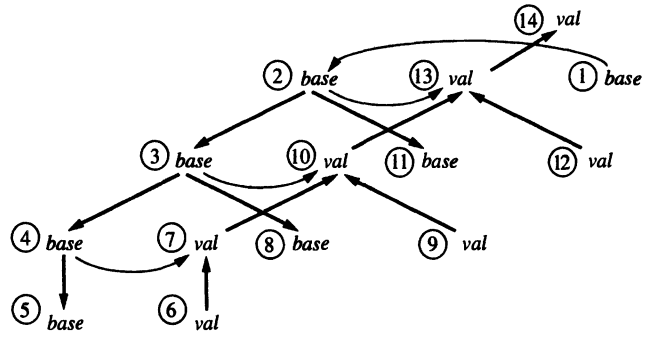


Figure 6.7
Dependency graph for the string 3450 (Example 6.9)

§

One question that arises in the use of a topological sort of the dependency graph to compute attribute values is how attribute values are found at the roots of the graph (a **root** of a graph is a node that has no predecessors). In Figure 6.7, nodes 1, 6, 9, and 12 are all roots of the graph.[6] Attribute values at these nodes do not depend on any other attribute, and so must be computed using information that is directly available. This information is often in the form of tokens that are children of the corresponding parse tree nodes. In Figure 6.7, for instance, the *val* of node 6 depends on the token **3** that is the child of the *digit* node that *val* corresponds to (see Figure 6.6). Thus, the attribute value at node 6 is 3. All such root values need to be computable prior to the computation of any other attribute values. Such computations are frequently performed by the scanner or the parser.

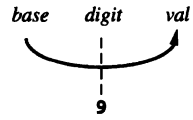
6. A root of the dependency graph should not be confused with the root of the parse tree.

It is possible to base an algorithm for attribute analysis on a construction of the dependency graph during compilation and a subsequent topological sort of the dependency graph to determine an order for the evaluation of the attributes. Since the construction of the dependency graph is based on the specific parse tree at compile time, this method is sometimes called a **parse tree method**. It is capable of evaluating the attributes in any attribute grammar that is **noncircular**, that is, an attribute grammar for which *every* possible dependency graph is acyclic.

There are several problems with this method. First, there is the added complexity required by the compile-time construction of the dependency graph. Second, while this method can determine whether a dependency graph is acyclic at compile time, it is generally inadequate to wait until compile time to discover a circularity, since a circularity almost certainly represents an error in the original attribute grammar. Instead, an attribute grammar should be tested in advance for noncircularity. There is an algorithm to do this (see the Notes and References section), but it is an exponential-time algorithm. Of course, this algorithm only needs to be run once, at compiler construction time, so this is not an overwhelming argument against it (at least for the purposes of compiler construction). The complexity of this approach is a more compelling argument.

The alternative to the above approach to attribute evaluation, and one adopted by practically every compiler, is for the compiler writer to analyze the attribute grammar and fix an order for attribute evaluation at compiler construction time. Although this method still uses the parse tree as a guide for attribute evaluation, the method is referred to as a **rule-based method**, since it depends on an analysis of the attribute equations, or semantic rules. Attribute grammars for which an attribute evaluation order can be fixed at compiler construction time form a class that is less general than the class of all noncircular attribute grammars, but in practice all reasonable attribute grammars have this property. They are sometimes called **strongly noncircular** attribute grammars. We proceed to a discussion of rule-based algorithms for this class of attribute grammars, after the following example.

Example 6.10 Consider again the dependency graphs of Example 6.8 and the topological sorts of the dependency graph discussed in Example 6.9 (see Figure 6.7). Even though nodes 6, 9, and 12 of Figure 6.7 are roots of the DAG, and thus all could occur at the beginning of a topological sort, in a rule-based method this is not possible. The reason is that any *val* might depend on the *base* of its associated *digit* node, if the corresponding token is **8** or **9**. For example, the dependency graph for *digit* → **9** is



Thus, in Figure 6.7, node 6 might have depended on node 5, node 9 might have depended on node 8, and node 12 might have depended on node 11. In a rule-based method, these nodes would be constrained to be evaluated after any nodes that they

might potentially depend on. Therefore, an evaluation order that, say, evaluates node 12 first (see Example 6.9), while correct for the particular tree of Figure 6.7, is not a valid order for a rule-based algorithm, since it would violate the order for other parse trees. §

6.2.2 Synthesized and Inherited Attributes

Rule-based attribute evaluation depends on an explicit or implicit traversal of the parse or syntax tree. Different kinds of traversals vary in power in terms of the kinds of attribute dependencies that can be handled. To study the differences, we must first classify attributes by the kinds of dependencies they exhibit. The simplest kind of dependency to handle is that of synthesized attributes, defined next.

Definition

An attribute is **synthesized** if all its dependencies point from child to parent in the parse tree. Equivalently, an attribute a is synthesized if, given a grammar rule $A \rightarrow X_1X_2 \ldots X_n$, the only associated attribute equation with an a on the left-hand side is of the form

$$A.a = f(X_1.a_1, \ldots, X_1.a_k, \ldots, X_n.a_1, \ldots, X_n.a_k)$$

An attribute grammar in which all the attributes are synthesized is called an **S-attributed grammar**.

We have already seen a number of examples of synthesized attributes and S-attributed grammars. The *val* attribute of numbers in Example 6.1 is synthesized (see the dependency graphs in Example 6.6, page 271), as is the *val* attribute of the simple integer arithmetic expressions in Example 6.2, page 264.

Given that a parse or syntax tree has been constructed by a parser, the attribute values of an S-attributed grammar can be computed by a single bottom-up, or postorder, traversal of the tree. This can be expressed by the following pseudocode for a recursive postorder attribute evaluator:

```
procedure PostEval ( T: treenode );
begin
  for each child C of T do
    PostEval ( C );
  compute all synthesized attributes of T;
end;
```

Example 6.11 Consider the attribute grammar of Example 6.2 for simple arithmetic expressions, with the synthesized attribute *val*. Given the following structure for a syntax tree (such as that of Figure 6.5, page 269)

```
typedef enum {Plus,Minus,Times} OpKind;
typedef enum {OpKind,ConstKind} ExpKind;
typedef struct streenode
      { ExpKind kind;
        OpKind op;
        struct streenode *lchild,*rchild;
         int val;
      } STreeNode;
typedef STreeNode *SyntaxTree;
```

the PostEval pseudocode would translate into the C code of Figure 6.8 for a left-to-right traversal.

Figure 6.8
C code for the postorder attribute evaluator for Example 6.11

```
void postEval(SyntaxTree t)
{ int temp;
  if (t->kind == OpKind)
  { postEval(t->lchild);
    postEval(t->rchild);
    switch (t->op)
    { case Plus:
        t->val = t->lchild->val + t->rchild->val;
        break;
      case Minus:
        t->val = t->lchild->val - t->rchild->val;
        break;
      case Times:
        t->val = t->lchild->val * t->rchild->val;
        break;
    } /* end switch */
  } /* end if */
} /* end postEval */
```

§

Not all attributes are synthesized, of course.

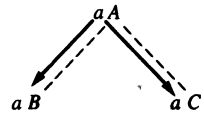
Definition

An attribute that is not synthesized is called an **inherited** attribute.

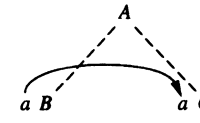
Examples of inherited attributes that we have already seen include the *dtype* attribute of Example 6.3 (page 265) and the *base* attribute of Example 6.4 (page 266). Inherited attributes have dependencies that flow either from parent to children in the parse tree (the reason for the name) or from sibling to sibling. Figures 6.9(a) and (b) illustrate the two basic kinds of dependency of inherited attributes. Both of these kinds of dependency occur in Example 6.7 for the *dtype* attribute. The reason that both are classified as inherited is that in algorithms to compute inherited attributes, sibling inheritance is often implemented in such a way that attribute values are passed from sibling

to sibling *via* the parent. Indeed, this is necessary if syntax tree edges only point from parent to child (a child thus cannot access its parent or siblings directly). On the other hand, if some structures in a syntax tree are implemented via sibling pointers, then sibling inheritance can proceed directly along a sibling chain, as depicted in Figure 6.9(c).

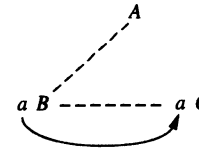
Figure 6.9
Different kinds of inherited dependencies



(a) Inheritance from parent to siblings



(b) Inheritance from sibling to sibling



(c) Sibling inheritance via sibling pointers

We turn now to algorithmic methods for the evaluation of inherited attributes. Inherited attributes can be computed by a preorder traversal, or combined preorder/inorder traversal of the parse or syntax tree. Schematically, this can be represented by the following pseudocode:

```
procedure PreEval ( T: treenode );
begin
  for each child C of T do
     compute all inherited attributes of C;
     PreEval ( C );
end;
```

Unlike synthesized attributes, the order in which the inherited attributes of the children are computed is important, since inherited attributes may have dependencies among the attributes of the children. The order in which each child C of T in the foregoing pseudocode is visited must therefore adhere to any requirements of these dependencies. In the next two examples, we demonstrate this using the *dtype* and *base* inherited attributes of previous examples.

Example 6.12 Consider the grammar of Example 6.3, which has the inherited attribute *dtype* and whose dependency graphs are given in Example 6.7, page 272 (see the attribute grammar in Table 6.3, page 266). Let us assume first that a parse tree has been explicitly constructed from the grammar, which for ease of reference we repeat here:

decl → *type var-list*
type → `int` | `float`
var-list → `id,` *var-list* | `id`

Then the pseudocode for a recursive procedure that computes the *dtype* attribute at all required nodes is as follows:

```
procedure EvalType ( T: treenode );
begin
  case nodekind of T of
  decl:
      EvalType ( type child of T );
      Assign dtype of type child of T to var-list child of T;
      EvalType ( var-list child of T );
  type:
      if child of T = int then T.dtype := integer
      else T.dtype := real;
  var-list:
      assign T.dtype to first child of T;
      if third child of T is not nil then
        assign T.dtype to third child;
        EvalType ( third child of T );
  end case;
end EvalType;
```

Notice how preorder and inorder operations are mixed, depending on the different kind of node being processed. For instance, a *decl* node requires that the *dtype* of its first child be computed first and then assigned to its second child before a recursive call of *EvalType* on that child; this is an in-order process. On the other hand, a *var-list* node assigns *dtype* to its children before making any recursive calls; this is a preorder process.

In Figure 6.10 we show the parse tree for the string `float x,y` together with the dependency graph for the *dtype* attribute, and we number the nodes in the order in which *dtype* is computed according to the above pseudocode.

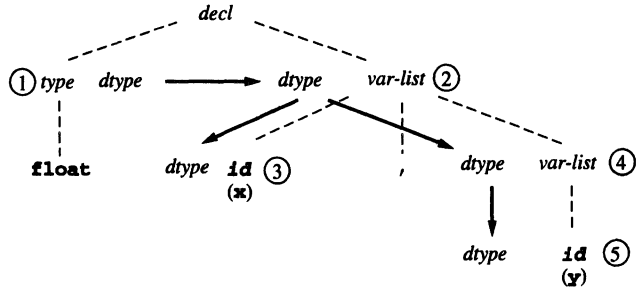
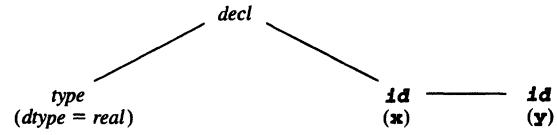


Figure 6.10
Parse tree showing traversal order for Example 6.12

To give a completely concrete form to this example, let us convert the previous pseudocode into actual C code. Also, rather than using an explicit parse tree, let us assume that a syntax tree has been constructed, in which *var-list* is represented by a sibling list of `id` nodes. Then a declaration string such as `float x,y` would have the syntax tree (compare this with Figure 6.10)



and the evaluation order of the children of the *decl* node would be left to right (the *type* node first, then the **x** node, and finally the **y** node). Note that in this tree we have already included the *dtype* of the *type* node; we assume this has been precomputed during the parse.

The syntax tree structure is given by the following C declarations:

```
typedef enum {decl,type,id} nodekind;
typedef enum {integer,real} typekind;
typedef struct treeNode
  { nodekind kind;
    struct treeNode
       * lchild, * rchild, * sibling;
    typekind dtype;
    /* for type and id nodes */
    char * name;
    /* for id nodes only */
  } * SyntaxTree;
```

The *EvalType* procedure now has the corresponding C code:

```
void evalType (SyntaxTree t)
{ switch (t->kind)
  { case decl:
      t->rchild->dtype = t->lchild->dtype;
      evalType(t->rchild);
      break;
    case id:
      if (t->sibling != NULL)
      { t->sibling->dtype = t->dtype;
        evalType(t->sibling);
      }
      break;
  } /* end switch */
} /* end evalType */
```

This code can be simplified to the following nonrecursive procedure, which operates entirely at the level of the root (**decl**) node:

```
void evalType (SyntaxTree t)
{ if (t->kind == decl)
  { SyntaxTree p = t->rchild;
    p->dtype = t->lchild->dtype;
    while (p->sibling != NULL)
    { p->sibling->dtype = p->dtype;
      p = p->sibling;
    }
  } /* end if */
} /* end evalType */
```

§

Example 6.13 Consider the grammar of Example 6.4 (page 266), which has the inherited attribute *base* (the dependency graphs are given in Example 6.8, page 273). We repeat the grammar of that example here:

based-num → *num basechar*
basechar → **o** | **d**
num → *num digit* | *digit*
digit → **0** | **1** | **2** | **3** | **4** | **5** | **6** | **7** | **8** | **9**

This grammar has two new features. First, there are two attributes, the synthesized attribute *val*, and the inherited attribute *base*, on which *val* depends. Second, the *base* attribute is inherited from the right child to the left child of a *based-num* (that is, from *basechar* to *num*). Thus, in this case, we must evaluate the children of a *based-num* from right to left instead of left to right. We proceed to give the pseudocode for an *EvalWithBase* procedure that computes both *base* and *val*. In this case, *base* is computed in preorder and *val* in postorder during a single pass (we discuss the question of multiple attributes and passes shortly). The pseudocode is as follows (see the attribute grammar in Table 6.4, page 267).

```
procedure EvalWithBase ( T: treenode );
begin
  case nodekind of T of
  based-num:
      EvalWithBase ( right child of T );
      assign base of right child of T to base of left child;
      EvalWithBase ( left child of T );
      assign val of left child of T to T.val;
  num:
      assign T.base to base of left child of T;
      EvalWithBase ( left child of T );
      if right child of T is not nil then
        assign T.base to base of right child of T;
        EvalWithBase ( right child of T );
        if vals of left and right children ≠ error then
            T.val := T.base*(val of left child) + val of right child
        else T.val := error;
      else T.val := val of left child;
```

```
    basechar:
        if child of T = o then T.base := 8
        else T.base := 10;
    digit:
        if T.base = 8 and child of T = 8 or 9 then T.val := error
        else T.val := numval ( child of T );
    end case;
end EvalWithBase;
```

We leave to the exercises the construction of C declarations for an appropriate syntax tree and a translation of the pseudocode for *EvalWithBase* into C code. §

In attribute grammars with combinations of synthesized and inherited attributes, if the synthesized attributes depend on the inherited attributes (as well as other synthesized attributes) but the inherited attributes do not depend on any synthesized attributes, then it is possible to compute all the attributes in a single pass over the parse or syntax tree. The previous example is a good example of how this is done, and the order of evaluation may be summarized by combining the *PostEval* and *PreEval* pseudocode procedures:

```
procedure CombinedEval ( T: treenode );
begin
  for each child C of T do
    compute all inherited attributes of C;
    CombinedEval ( C );
  compute all synthesized attributes of T;
end;
```

Situations in which inherited attributes depend on synthesized attributes are more complex and require more than one pass over the parse or syntax tree, as the following example shows.

Example 6.14 Consider the following simple version of an expression grammar:

$$exp \rightarrow exp \ / \ exp \mid \mathbf{num} \mid \mathbf{num.num}$$

This grammar has a single operation, division, indicated by the token `/`. It also has two versions of numbers: integer numbers consisting of sequences of digits, which we indicate by the token **num**, and floating-point numbers, which we indicate by the token **num.num**. The idea of this grammar is that operations may be interpreted differently, depending on whether they are floating-point or strictly integer operations. Division, in particular, is quite different, depending on whether fractions are allowed. If not, division is often called a **div** operation, and the value of `5/4` is 5 **div** 4 = 1. If floating-point division is meant, then `5/4` has value 1.2.

Suppose now that a programming language requires mixed expressions to be promoted to floating-point expressions throughout, and the appropriate operations to be used in their semantics. Thus, the meaning of the expression `5/2/2.0` (assuming left

associativity of division) is 1.25, while the meaning of **5/2/2** is 1.[7] To describe these semantics requires three attributes: a synthesized Boolean attribute *isFloat* which indicates if any part of an expression has a floating-point value; an inherited attribute *etype*, with two values *int* and *float*, which gives the type of each subexpression and which depends on *isFloat*; and, finally, the computed *val* of each subexpression, which depends on the inherited *etype*. This situation also requires that the top-level expression be identified (so that we know there are no more subexpressions to be considered). We do this by augmenting the grammar with a start symbol:

$$S \rightarrow exp$$

The attribute equations are given in Table 6.7. In the equations for the grammar rule $exp \rightarrow$ **num**, we have used *Float*(**num**.*val*) to indicate a function that converts the integer value **num**.*val* into a floating-point value. We have also used / for floating-point division and **div** for integer division.

The *isFloat*, *etype*, and *val* attributes in this example can be computed by two passes over the parse or syntax tree. The first pass computes the synthesized attribute *isFloat* by a postorder traversal. The second pass computes both the inherited attribute *etype* and the synthesized attribute *val* in a combined preorder/postorder traversal. We leave the description of these passes and the corresponding attribute computations for the expression 5/2/2.0 to the exercises, as well as the construction of pseudocode or C code to perform the successive passes on the syntax tree.

Table 6.7
Attribute grammar for Example 6.14

Grammar Rule	Semantic Rules
$S \rightarrow exp$	*exp.etype* = **if** *exp.isFloat* **then** *float* **else** *int* *S.val* = *exp.val*
$exp_1 \rightarrow exp_2$ / exp_3	exp_1.*isFloat* = exp_2.*isFloat* **or** exp_3.*isFloat* exp_2.*etype* = exp_1.*etype* exp_3.*etype* = exp_1.*etype* exp_1.*val* = **if** exp_1.*etype* = *int* **then** exp_2.*val* **div** exp_3.*val* **else** exp_2.*val* / exp_3.*val*
$exp \rightarrow$ **num**	*exp.isFloat* = **false** *exp.val* = **if** *exp.etype* = *int* **then** **num**.*val* **else** *Float*(**num**.*val*)
$exp \rightarrow$ **num.num**	*exp.isFloat* = **true** *exp.val* = **num.num**.*val*

§

7. This rule is not the same rule as the one used in C. For instance, the value of **5/2/2.0** is 1.0 in C. not 1.25.

6.2.3 Attributes as Parameters and Returned Values

Often when computing attributes, it makes sense to use parameters and returned function values to communicate attribute values rather than to store them as fields in a syntax tree record structure. This is particularly true if many of the attribute values are the same or are only used temporarily to compute other attribute values. In this case, it makes little sense to use space in the syntax tree to store attribute values at each node. A single recursive traversal procedure that computes inherited attributes in preorder and synthesized attributes in postorder can, in fact, pass the inherited attribute values as parameters to recursive calls on children and receive synthesized attribute values as returned values of those same calls. Several examples of this have already occurred in previous chapters. In particular, the computation of the synthesized value of an arithmetic expression can be done by a recursive parsing procedure that returns the value of the current expression. Similarly, the syntax tree as a synthesized attribute must itself be computed by returned value during parsing, since until it has been constructed, no data structure yet exists in which to record itself as an attribute.

In more complex situations, for instance when more than one synthesized attribute must be returned, it may be necessary to use a record structure or union as a returned value, or one may split up the recursive procedure into several procedures to cover different cases. We illustrate this with an example.

Example 6.15 Consider the recursive procedure *EvalWithBase* of Example 6.13. In this procedure, the *base* attribute of a number is computed only once and then is used for all subsequent computations of the *val* attribute. Similarly, the *val* attribute of a part of the number is only used as a temporary in the computation of the value of the complete number. It makes sense to turn *base* into a parameter (as an inherited attribute) and *val* into a returned value. The modified *EvalWithBase* procedure is as follows:

```
function EvalWithBase ( T: treenode; base: integer ): integer;
var temp, temp2: integer;
begin
  case nodekind of T of
  based-num:
      temp := EvalWithBase ( right child of T );
      return EvalWithBase ( left child of T, temp );
  num:
      temp := EvalWithBase ( left child of T, base );
      if right child of T is not nil then
        temp2 := EvalWithBase ( right child of T, base );
        if temp ≠ error and temp2 ≠ error then
            return base*temp + temp2
        else return error;
      else return temp;
  basechar:
      if child of T = o then return 8
      else return 10;
  digit:
      if base = 8 and child of T = 8 or 9 then return error
      else return numval ( child of T );
  end case;
end EvalWithBase;
```

Of course, this only works because the *base* attribute and the *val* attribute have the same *integer* data type, since in one case *EvalWithBase* returns the *base* atttribute (when the parse tree node is a *basechar* node), and in the other cases *EvalWithBase* returns the *val* attribute. It is also somewhat irregular that the first call to *EvalWithBase* (on the root *based_num* parse tree node) must have a *base* value supplied, even though one does not yet exist, and which is subsequently ignored. For example, to start the computation, one would have to make a call such as

EvalWithBase(*rootnode*,0);

with a dummy base value 0. It would be more rational, therefore, to distinguish three cases—the *based_num* case, the *basechar* case, and the *num* and *digit* case—and write three separate procedures for these three cases. The pseudocode would then appear as follows:

```
function EvalBasedNum ( T: treenode ): integer;
(* only called on root node *)
begin
   return EvalNum ( left child of T, EvalBase(right child of T) );
end EvalBasedNum;

function EvalBase ( T: treenode ): integer;
(* only called on basechar node *)
begin
   if child of T = 0 then return 8
   else return 10;
end EvalBase;

function EvalNum ( T: treenode; base: integer ): integer;
var temp, temp2: integer;
begin
   case nodekind of T of
   num:
         temp := EvalWithBase ( left child of T, base );
         if right child of T is not nil then
            temp2 := EvalWithBase ( right child of T, base );
            if temp ≠ error and temp2 ≠ error then
                  return base*temp + temp2
            else return error;
         else return temp;
   digit:
         if base = 8 and child of T = 8 or 9 then return error
         else return numval ( child of T );
   end case;
end EvalNum;
```

§

6.2.4 The Use of External Data Structures to Store Attribute Values

In those cases where attribute values do not lend themselves easily to the method of parameters and returned values (particularly true when attribute values have significant

structure and may be needed at arbitrary points during translation), it may still not be reasonable to store attribute values in the syntax tree nodes. In such cases, data structures such as lookup tables, graphs, and other structures may be useful to obtain the correct behavior and accessibility of attribute values. The attribute grammar itself can be modified to reflect this need by replacing attribute equations (representing assignments of attribute values) by calls to procedures representing operations on the appropriate data structures used to maintain the attribute values. The resulting semantic rules then no longer represent an attribute grammar, but are still useful in describing the semantics of the attributes, as long as the operation of the procedures is made clear.

Example 6.16 Consider the previous example with the *EvalWithBase* procedure using parameters and returned values. Since the *base* attribute, once it is set, is fixed for the duration of the value computation, we may use a nonlocal variable to store its value, rather than passing it as a parameter each time. (If *base* were not fixed, this would be risky or even incorrect in such a recursive process.) Thus, we can alter the pseudocode for *EvalWithBase* as follows:

```
function EvalWithBase ( T: treenode ): integer;
var temp, temp2: integer;
begin
  case nodekind of T of
  based-num:
       SetBase ( right child of T );
       return EvalWithBase ( left child of T );
  num:
       temp := EvalWithBase ( left child of T );
       if right child of T is not nil then
          temp2 := EvalWithBase ( right child of T );
          if temp ≠ error and temp2 ≠ error then
               return base*temp + temp2
          else return error;
       else return temp;
  digit:
       if base = 8 and child of T = 8 or 9 then return error
       else return numval ( child of T );
  end case;
end EvalWithBase;

procedure SetBase ( T: treenode );
begin
  if child of T = 0 then base := 8
  else base := 10;
end SetBase ;
```

Here we have separated out the process of assigning to the nonlocal *base* variable in the procedure *SetBase*, which is only called on a *basechar* node. The rest of the code of *EvalWithBase* then simply refers to *base* directly, without passing it as a parameter.

We can also change the semantic rules to reflect the use of the nonlocal *base* variable. In this case, the rules would look something like the following, where we have used assignment to explicitly indicate the setting of the nonlocal *base* variable:

Grammar Rule	Semantic Rules
based-num → *num basechar*	*based-num.val* = *num.val*
basechar → **o**	*base* := 8
basechar → **d**	*base* := 10
num_1 → num_2 *digit*	num_1.*val* = **if** *digit.val* = *error* **or** num_2.*val* = error **then** *error* **else** num_2.*val* * *base* + *digit.val*
etc.	etc.

Now *base* is no longer an attribute in the sense in which we have used it up to now, and the semantic rules no longer form an attribute grammar. Nevertheless, if *base* is known to be a variable with the appropriate properties, then these rules still adequately define the semantics of a *based-num* for the compiler writer. §

One of the prime examples of a data structure external to the syntax tree is the **symbol table**, which stores attributes associated to declared constants, variables, and procedures in a program. A symbol table is a dictionary data structure with operations such as *insert*, *lookup*, and *delete*. In the next section we discuss issues surrounding the symbol table in typical programming languages. We content ourselves in this section with the following simple example.

Example 6.17 Consider the attribute grammar of simple declarations of Table 6.3 (page 266), and the attribute evaluation procedure for this attribute grammar given in Example 6.12 (page 279). Typically, the information in declarations is inserted into a symbol table using the declared identifiers as keys and stored there for later lookup during the translation of other parts of the program. Let us, therefore, assume for this attribute grammar that a symbol table exists that will store the identifier name together with its declared data type and that name-data type pairs are inserted into the symbol table via an *insert* procedure declared as follows:

procedure *insert* (*name*: *string*; *dtype*: *typekind*);

Thus, instead of storing the data type of each variable in the syntax tree, we insert it into the symbol table using this procedure. Also, since each declaration has only one associated type, we can use a nonlocal variable to store the constant *dtype* of each declaration during processing. The resulting semantic rules are as follows:

Grammar Rule	Semantic Rules
decl → *type var-list*	
type → **int**	*dtype* = *integer*
type → **float**	*dtype* = *real*
$var\text{-}list_1$ → **id** , $var\text{-}list_2$	*insert*(**id**.*name*, *dtype*)
var-list → **id**	*insert*(**id**.*name*, *dtype*)

In the calls to *insert* we have used `id` *.name* to refer to the identifier string, which we assume to be computed by the scanner or parser. These semantic rules are quite different from the corresponding attribute grammar; the grammar rule for a *decl* has, in fact, no semantic rules at all. Dependencies are not as clearly expressed, although it is clear that the *type* rules must be processed before the associated *var-list* rules, since the calls to *insert* depend on *dtype*, which is set in the *type* rules.

The corresponding pseudocode for an attribute evaluation procedure *EvalType* is as follows (compare this with the code on page 280):

```
procedure EvalType ( T: treenode );
begin
  case nodekind of T of
  decl:
       EvalType ( type child of T );
       EvalType ( var-list child of T );
  type:
       if child of T = int then dtype := integer
       else dtype := real;
  var-list:
       insert(name of first child of T, dtype)
       if third child of T is not nil then
          EvalType ( third child of T );
  end case;
end EvalType;
```

§

6.2.5 The Computation of Attributes During Parsing

A question that naturally occurs is to what extent attributes can be computed at the same time as the parsing stage, without waiting to perform further passes over the source code by recursive traversals of the syntax tree. This is particularly important with regard to the syntax tree itself, which is a synthesized attribute that must be constructed during the parse, if it is to be used for further semantic analysis. Historically, the possibility of computing all attributes during the parse was of even greater interest, since considerable emphasis was placed on the capability of a compiler to perform one-pass translation. Nowadays this is less important, so we do not provide an exhaustive analysis of all the special techniques that have been developed. However, a basic overview of the ideas and requirements is worthwile.

Which attributes can be successfully computed during a parse depends to great extent on the power and properties of the parsing method employed. One important restriction is provided by the fact that all the major parsing methods process the input program from left to right (this is the content of the first L in the LL and LR parsing techniques studied in the previous two chapters). This is equivalent to the requirement that the attributes be capable of evaluation by a left-to-right traversal of the parse tree. For synthesized attributes this is not a restriction, since the children of a node can be processed in arbitrary order, in particular from left to right. But, for inherited attributes, this means that there may be no "backward" dependencies in the dependency graph (dependencies pointing from right to left in the parse tree). For example, the attribute grammar of Example 6.4 (page 266) violates this property, since a *based-num* has its

base given by the suffix **o** or **d**, and the *val* attribute cannot be computed until the suffix is seen and processed at the end of the string. Attribute grammars that *do* satisfy this property are called L-attributed (for left to right), and we make the following definition.

Definition

An attribute grammar for attributes $a_1, \ldots, a_k$ is **L-attributed** if, for each inherited attribute a_j and each grammar rule

$$X_0 \to X_1 X_2 \ldots X_n$$

the associated equations for a_j are all of the form

$$X_i.a_j = f_{ij}(X_0.a_1, \ldots, X_0.a_k, X_1.a_1, \ldots, X_1.a_k, \ldots, X_{i-1}.a_1, \ldots, X_{i-1}.a_k)$$

That is, the value of a_j at X_i can only depend on attributes of the symbols $X_0, \ldots, X_{i-1}$ that occur to the left of X_i in the grammar rule.

As a special case, we have already noted that an S-attributed grammar is L-attributed.

Given an L-attributed grammar in which the inherited attributes do not depend on the synthesized attributes, a recursive-descent parser can evaluate all the attributes by turning the inherited attributes into parameters and synthesized attributes into returned values, as previously described. Unfortunately, LR parsers, such as a Yacc-generated LALR(1) parser, are suited to handling primarily synthesized attributes. The reason lies, ironically, in the greater power of LR parsers over LL parsers. Attributes only become computable when the grammar rule to be used in a derivation becomes known, since only then are the equations for the attribute computation determined. LR parsers, however, put off deciding which grammar rule to use in a derivation until the right-hand side of a grammar rule is fully formed. This makes it difficult for inherited attributes to be made available, unless their properties remain fixed for all possible right-hand side choices. We will briefly discuss the use of the parsing stack to compute attributes in the most common cases, with applications to Yacc. Sources for more complex techniques are mentioned in the Notes and References section.

Computing Synthesized Attributes During LR Parsing This is the easy case for an LR parser. An LR parser will generally have a **value stack** in which the synthesized attributes are stored (perhaps as unions or structures, if there is more than one attribute for each grammar symbol). The value stack will be manipulated in parallel with the parsing stack, with new values being computed according to the attribute equations each time a shift or a reduction on the parsing stack occurs. We illustrate this in Table 6.8 for the attribute grammar of Table 6.5 (page 269), which is the ambiguous version of the simple arithmetic expression grammar. For simplicity, we use abbreviated notation for the grammar and ignore some of the details of an LR parsing algorithm in the table. In par-

ticular, we do not indicate state numbers, we do not show the augmented start symbol, and we do not express the implicit disambiguating rules. The table contains two new columns in addition to the usual parsing actions: the value stack and semantic actions. The semantic actions indicate how computations occur on the value stack as reductions occur on the parsing stack. (Shifts are viewed as pushing the token values on both the parsing stack and the value stack, though this may differ in individual parsers.)

As an example of a semantic action, consider step 10 in Table 6.8. The value stack contains the integer values 12 and 5, separated by the token +, with 5 at the top of the stack. The parsing action is to reduce by $E \rightarrow E + E$, and the corresponding semantic action from Table 6.5 is to compute according to the equation $E_1.val = E_2.val + E_3.val$. The corresponding actions on the value stack taken by the parser are as follows (in pseudocode):

```
pop t3          { get E3.val from the value stack }
pop             { discard the + token }
pop t2          { get E2.val from the value stack }
t1 = t2 + t3    { add }
push t1         { push the result back onto the value stack }
```

In Yacc the situation represented by the reduction at step 10 would be written as the rule

```
E : E + E { $$ = $1 + $3; }
```

Here the pseudovariables `$i` represent positions in the right-hand side of the rule being reduced and are converted to value stack positions by counting backward from the right. Thus, `$3`, corresponding to the rightmost `E`, is to be found at the top of the stack, while `$1` is to be found two positions below the top.

Table 6.8 Parsing and semantic actions for the expression `3*4+5` during an LR parse

	Parsing Stack	Input	Parsing Action	Value Stack	Semantic Action
1	$	3*4+5 $	shift	$	
2	$ n	*4+5 $	reduce $E \rightarrow$ n	$ n	$E.val = n.val$
3	$ E	*4+5 $	shift	$ 3	
4	$ E *	4+5 $	shift	$ 3 *	
5	$ E * n	+5 $	reduce $E \rightarrow$ n	$ 3 * n	$E.val = n.val$
6	$ E * E	+5 $	reduce $E \rightarrow E * E$	$ 3 * 4	$E_1.val = E_2.val * E_3.val$
7	$ E	+5 $	shift	$ 12	
8	$ E +	5 $	shift	$ 12 +	
9	$ E + n	$	reduce $E \rightarrow$ n	$ 12 + n	$E.val = n.val$
10	$ E + E	$	reduce $E \rightarrow E + E$	$ 12 + 5	$E_1.val = E_2.val + E_3.val$
11	$ E	$		$ 17	

Inheriting a Previously Computed Synthesized Attribute During LR Parsing Because of the left-to-right evaluation strategy of LR parsing, an action associated to a nonterminal in the right-hand side of a rule can make use of synthesized attributes of the symbols to the left of it in the rule, since these values have already been pushed onto the value stack. To illustrate this briefly, consider the production choice $A \rightarrow B\ C$, and suppose that C has an inherited attribute i that depends in some way on the synthesized attribute s of B: $C.i = f(B.s)$. The value of $C.i$ can be stored in a variable prior to the recognition of C by introducing an ε-production between B and C that schedules the storing of the top of the value stack:

Grammar Rule	Semantic Rules
$A \rightarrow B\ D\ C$	
$B \rightarrow \ldots$	{ compute $B.s$ }
$D \rightarrow \varepsilon$	$saved\_i = f(valstack[top])$
$C \rightarrow \ldots$	{ now $saved\_i$ is available }

In Yacc this process is made even easier, since the ε-production need not be introduced explicitly. Instead, the action of storing the computed attribute is simply written at the place in the rule where it is to be scheduled:

```
A : B { saved_i = f($1); } C ;
```

(Here the pseudovariable **$1** refers to the value of **B**, which is at the top of the stack when the action is executed.) Such embedded actions in Yacc were studied in Section 5.5.6 of the previous chapter.

An alternative to this strategy is available when the position of a previously computed synthesized attribute in the value stack can always be predicted. In this case, the value need not be copied into a variable, but can be accessed directly on the value stack. Consider, for example, the following L-attributed grammar with an inherited *dtype* attribute:

Grammar Rule	Semantic Rules
decl → *type var-list*	*var-list.dtype* = *type.dtype*
type → **int**	*type.dtype* = *integer*
type → **float**	*type.dtype* = *real*
$var\text{-}list_1 \rightarrow var\text{-}list_2$ **, id**	*insert*(**id**.*name*, $var\text{-}list_1$.*dtype*)
	$var\text{-}list_2.dtype = var\text{-}list_1.dtype$
var-list → **id**	*insert*(**id**.*name*, *var-list.dtype*)

In this case, the *dtype* attribute, which is a synthesized attribute for the *type* nonterminal, can be computed onto the value stack just before the first *var-list* is recognized. Then, when each rule for *var-list* is reduced, *dtype* can be found at a fixed position in the value stack by counting backward from the top: when *var-list* → **id** is reduced, *dtype* is just below the stack top, while when $var\text{-}list_1 \rightarrow var\text{-}list_2$ **,id** is reduced, *dtype* is three positions below the top of the stack. We can implement this in an LR parser by

eliminating the two **copy rules** for *dtype* in the above attribute grammar and accessing the value stack directly:

Grammar Rule	Semantic Rules
decl → *type var-list*	
type → `int`	*type.dtype* = *integer*
type → `float`	*type.dtype* = *real*
$var\text{-}list_1$ → $var\text{-}list_2$ `,id`	*insert*(`id`.*name*, *valstack*[*top*−3])
var-list → `id`	*insert*(`id`.*name*, *valstack*[*top*−1])

(Note that, since *var-list* has no synthesized *dtype* attribute, the parser must push a dummy value onto the stack to retain the proper placement in the stack.)

There are several problems with this method. First, it requires the programmer to directly access the value stack during a parse, and this may be risky in automatically generated parsers. Yacc, for instance, has no pseudovariable convention for accessing the value stack below the current rule being recognized, as would be required by the above method. Thus, to implement such a scheme in Yacc, one would have to write special code to do this. The second problem is that this technique only works if the position of the previously computed attribute is predictable from the grammar. If, for example, we had written the above declaration grammar so that *var-list* was right recursive (as we did in Example 6.17), then an arbitrary number of `id`'s could be on the stack, and the position of *dtype* in the stack would be unknown.

By far the best technique for dealing with inherited attributes in LR parsing is to use external data structures, such as a symbol table or nonlocal variables, to hold inherited attribute values, and to add ε-productions (or embedded actions as in Yacc) to allow for changes to these data structures to occur at appropriate moments. For example, a solution to the *dtype* problem just discussed can be found in the discussion of embedded actions in Yacc (Section 5.5.6).

We should be aware, however, that even this latter method is not without pitfalls: adding ε-productions to a grammar can add parsing conflicts, so that an LALR(1) grammar can turn into a non-LR(k) grammar for any k (see Exercise 6.15 and the Notes and References section). In practical situations, this rarely happens, however.

6.2.6 The Dependence of Attribute Computation on the Syntax

As the final topic in this section, it is worth noting that the properties of attributes depend heavily on the structure of the grammar. It can happen that modifications to the grammar that do not change the legal strings of the language can make the computation of attributes simpler or more complex. Indeed, we have the following:

Theorem

(From Knuth [1968]). Given an attribute grammar, all inherited attributes can be changed into synthesized attributes by suitable modification of the grammar, without changing the language of the grammar.

We give one example of how an inherited attribute can be turned into a synthesized attribute by modification of the grammar.

Example 6.18 Consider the grammar of simple declarations considered in the previous examples:

$$\begin{aligned} &\textit{decl} \rightarrow \textit{type var-list} \\ &\textit{type} \rightarrow \texttt{int} \mid \texttt{float} \\ &\textit{var-list} \rightarrow \texttt{id , } \textit{var-list} \mid \texttt{id} \end{aligned}$$

The *dtype* attribute of the attribute grammar of Table 6.3 is inherited. If, however, we rewrite the grammar as follows,

$$\begin{aligned} &\textit{decl} \rightarrow \textit{var-list } \texttt{id} \\ &\textit{var-list} \rightarrow \textit{var-list } \texttt{id , } \mid \textit{type} \\ &\textit{type} \rightarrow \texttt{int} \mid \texttt{float} \end{aligned}$$

then the same strings are accepted, but the *dtype* attribute now becomes synthesized, according to the following attribute grammar:

Grammar Rule	Semantic Rules
$\textit{decl} \rightarrow \textit{var-list } \texttt{id}$	$\texttt{id}.dtype = \textit{var-list}.dtype$
$\textit{var-list}_1 \rightarrow \textit{var-list}_2 \texttt{ id ,}$	$\textit{var-list}_1.dtype = \textit{var-list}_2.dtype$
	$\texttt{id}.dtype = \textit{var-list}_1.dtype$
$\textit{var-list} \rightarrow \textit{type}$	$\textit{var-list}.dtype = \textit{type}.dtype$
$\textit{type} \rightarrow \texttt{int}$	$\textit{type}.dtype = \textit{integer}$
$\textit{type} \rightarrow \texttt{float}$	$\textit{type}.dtype = \textit{real}$

We illustrate how this change in the grammar affects the parse tree and the *dtype* attribute computation in Figure 6.11, which shows the parse tree for the string **float x,y**, together with the attribute values and dependencies. The dependencies of the two **id** .*dtype* values on the values of the parent or sibling are drawn as dashed lines in the figure. While these dependencies appear to be violations of the claim that there are no inherited attributes in this attribute grammar, in fact these dependencies are always to leaves in the parse tree (that is, not recursive) and may be achieved by operations at the appropriate parent nodes. Thus, these dependencies are not viewed as inheritances. §

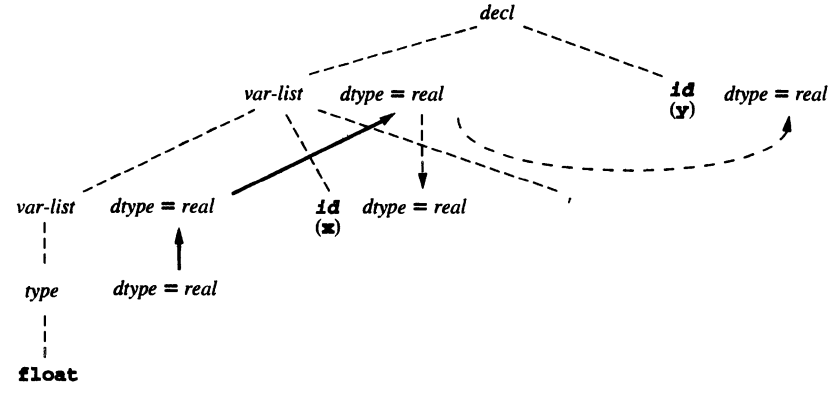


Figure 6.11 Parse tree for the string **float x,y** showing the *dtype* attribute as specified by the attribute grammar of Example 6.18

In fact, the stated theorem is less useful than it might seem. Changing the grammar in order to turn inherited attributes into synthesized attributes often makes the grammar and the semantic rules much more complex and difficult to understand. It is, therefore, not a recommended way to deal with the problems of computing inherited attributes. On the other hand, if an attribute computation seems unnaturally difficult, it may be because the grammar is defined in an unsuitable way for its computation, and a change in the grammar may be worthwhile.

6.3 THE SYMBOL TABLE

The symbol table is the major inherited attribute in a compiler and, after the syntax tree, forms the major data structure as well. Although we have, with a few exceptions, delayed a discussion of the symbol table to this point, where it fits best into the conceptual framework of the semantic analysis phase, the reader should be aware that in practical compilers the symbol table is often intimately involved with the parser and even the scanner, either of which may need to enter information directly into the symbol table or consult it to resolve ambiguities (for one such example from C, see Exercise 6.22). Nevertheless, in a very carefully designed language such as Ada or Pascal, it is possible and even reasonable to put off symbol table operations until after a complete parse, when the program being translated is known to be syntactically correct. We have done this, for example, in the TINY compiler, whose symbol table will be studied later in this chapter.

The principal symbol table operations are *insert*, *lookup*, and *delete*; other operations may also be necessary. The *insert* operation is used to store the information provided by name declarations when processing these declarations. The *lookup* operation is needed to retrieve the information associated to a name when that name is used in the associated code. The *delete* operation is needed to remove the information provided by a declaration when that declaration no longer applies.[8] The properties of these operations are dictated by the rules of the programming language being translated. In particular, what information needs to be stored in the symbol table is a function of the structure and purpose of the declarations. Typically this includes data type information, information on region of applicability (scope, discussed below), and information on eventual location in memory.

In this section we discuss, first, the organization of the symbol table data structure for speed and ease of access. Subsequently, we describe some typical language requirements and the effect they have on the operation of the symbol table. Last, we give an extended example of the use of a symbol table with an attribute grammar.

6.3.1 The Structure of the Symbol Table

The symbol table in a compiler is a typical dictionary data structure. The efficiency of the three basic operations *insert*, *lookup*, and *delete* vary according to the organization of the data structure. The analysis of this efficiency for different organizations, and the

8. Rather than destroying this information, a *delete* operation is more likely to remove it from view, either by storing it elsewhere, or by marking it as no longer active.